

# **TreatmentFlow:**

## **Finding the Optimal Patient Triage System for Hospital Usage**



[Graphic above was generated by AI]

By Emese Elkind and Adam Neto

CISC 352: Artificial Intelligence

Professor Christian Muise

Queen's University School of Computing

## Abstract

Ontario emergency departments have an average wait time of 2.2 hours, with triage nurses responsible for assigning hospital resources to patients from priority levels 1–5 based on urgency—a role known to be highly stressful. To address this, we propose *TreatmentFlow*, an AI-based digital solution for optimizing the flow of patients through the emergency department. We introduce *TreatmentFlow*, an AI-driven system designed to improve emergency patient flow through automated triage, bed assignment, and diagnosis support systems. Our Deep Learning model predicts triage priority with 66% accuracy, showing promise but requiring improvement. For bed assignments, a Greedy Constraint Optimization approach efficiently minimizes delays, outperforming more complex constraint programming. A Bayesian Network approach achieves perfect accuracy in classifying pre-defined diagnoses, though real-time input limitations persist. Together, these modules aim to streamline ED operations, ease nurse workload, and assist physicians. While AI tools show clear potential in healthcare, challenges remain in maximizing accuracy and minimizing computational complexity.

## I. Introduction

**H**EALTHCARE is constantly evolving. With emergency room wait times averaging 2.2 hours in Ontario [1], Artificial-Intelligence-driven triage systems could be the key to prioritizing patients more efficiently and reducing delays. Triage is the process hospitals use to determine the urgency of treatment for each patient [2]. This is determined by the patient's conditions and symptoms when they enter the hospital. Triage ensures that patients needing urgent care wait the least amount of time for treatment. In Ontario, patients are assigned a Canadian Triage and Acuity Scale (CTAS) priority number from 1-5 based on urgency of care (Table 1):

**Table 1:** Canadian Triage and Acuity Scale (CTAS) priority leveling [3].

| CTAS Levels | Description   |
|-------------|---------------|
| 1           | Resuscitation |
| 2           | Emergent      |
| 3           | Urgent        |
| 4           | Less Urgent   |
| 5           | Non-Urgent    |

Triage nurses check vital signs, symptoms, and gather patient history data to assess where each patient fits on this priority list. This is known to be one of the most stressful positions a nurse can have in the hospital [4]. Today, hospitals are increasingly using technology to support triage efforts and reduce pressure on nurses, with artificial intelligence (AI) and machine learning (ML) emerging as innovative tools in this area.

We propose *TreatmentFlow*, an AI-based solution for optimizing the flow of patients through the emergency room. We examine three problems in triage and apply AI techniques to solve them. Firstly, we aim to automate the assignment of patient priority using triage symptom information, relieving some triage decision-making pressure for nurses. The method presented to solve this is a Deep Learning approach. In this approach, we use a Deep Neural Network model trained on a large real-world dataset to predict the CTAS priority level of incoming patients given each patient’s symptom data.

Next, we seek to optimally assign patients to beds based on patient priority, further simplifying the decision-making tasks of the nurses. This is approached with two different methods: Mixed Integer Programming (MIP) Constraint Optimization, and Greedy patient assignment. In the MIP approach, we construct a model of variables and constraints to mimic a hospital and optimize (minimize) the total wait time of patients, weighted by priority level. This weighting implies that the model considers that the wait times of high-priority patients are more detrimental than the wait times of lower-priority patients. To test the value of this MIP solution, we compare its results to those of a greedy patient assignment algorithm. This comparison evaluates either solution’s performance and efficiency at scale, including objective (sum of weighted wait times) minimization, and execution time.

Finally, our solution involves utilizing probabilistic inference and Bayesian Networks to produce a probability table for each patient, which displays potential diagnoses given a set of symptoms. Doctors in the emergency department often ask for each other’s opinions to ensure correct diagnosis. Double-checking diagnoses is important but can sometimes waste time and be ineffective. We propose a more efficient solution: a Bayesian Network decision support tool to predict possible diagnoses and produce a doctors-only bedside document that includes the probability of each predicted diagnoses. This aids in the efficient, correct diagnosis of patients. By providing a decision support tool, doctors will be able to more efficiently deliberate between methods of treatment.

## II. Methodology

Two datasets containing patient symptom information were used to train the Deep Learning and Bayesian Network components of *TreatmentFlow*: the *Hospital Triage and Patient History Data* from Yale New Haven Health System, covering March 2014 to July 2017, and *SymbiPredict Symptom-Disease Prediction Dataset (SDPD)*, sourced from Kaggle [5] and Mendeley [6], respectively.

The Yale dataset includes approximately 50,000 patient records with 972 variables, including patient information and symptoms. To manage file size and enable efficient analysis, the original 103.45 MB RData file was converted to CSV format and down sampled by selecting every 75th

row, resulting in a reduced file size of 19.8 MB ensuring some level of random sampling while maintaining a manageable dataset size for analysis.

The *SymbiPredict* dataset, comprising nearly 5,000 symptom–prognosis pairs in CSV format, required no preprocessing due to its smaller size (1.32 MB) and clean structure.

It is important to note that both datasets originate from U.S. healthcare systems, where Emergency Severity Index (ESI) is used for patient triage instead of the Canadian CTAS scale. These two priority measures represent the same scale and thus are effectively equivalent. Therefore, the python scripts reference ESI while this paper refers to CTAS for consistency with the Canadian healthcare context.

### A. Constraint Satisfaction

The Constraint Satisfaction component of *TreatmentFlow* is a set of solutions for hospital bed assignment based on incoming patients and available resources. The two solutions are (1) a mixed integer programming (MIP) constraint optimization solution and (2) a greedy bed assignment solution. These solutions are compared in terms of computation efficiency and quality of results.

Both solutions rely on a system of hospital objects formed in the file “HospitalClasses.py”. The necessary objects include Bed, Patient, and HospitalRecords, each of which store and track information for instances of their category. For example, each Bed object stores their occupant (if any), each Patient stores statistics like CTAS priority level and arrival time, and HospitalRecords stores a database of patients. In our first iteration of *TreatmentFlow*’s constraint satisfaction component, we track the *inverse* of CTAS priorities, meaning we treat 5 as the highest priority and 1 as the lowest to simplify current calculations. We also use a randomly generated set of patients, arrival times, and priorities, whereas we chain all three components together for a given set of patients in our final product.

In the greedy solution to hospital bed assignment found in the “GreedyBedAssignment.py” file, we examine the current state of the hospital minute-by-minute over a 24-hour period of with the assistance of custom HospitalRecords attributes such as “unserved” (the priority queue of unserved patients) and “serving” (the running list of patients occupying a bed). For each minute of the simulation, the scheduler assigns the next available highest-priority patient to any available bed and keeps them there for the duration of their service time. This assignment considers the patient’s CTAS priority level (inverse) and arrival time at the hospital. Given the number of patients ( $p$ ) and number of beds ( $b$ ), the hospital simulator method “run\_hospital()” features a time complexity of  $O(p * b)$ , which is very efficient at scale. While this approach is algorithmically efficient, it is naïve in the fact that it does not guarantee an optimal solution but instead assigns patients based on currently available information.

Constraint optimization problems involve modeling a set of variables and assigning them values such that all constraints on variables are satisfied. They then use this model to optimize for a given objective function, which is either minimized or maximized.

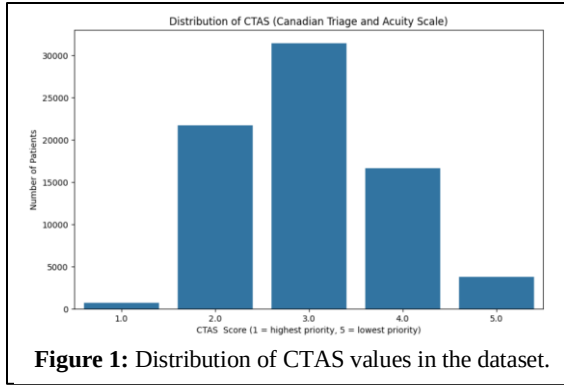
In the mixed integer programming (MIP) approach to hospital bed assignment found in the “MIPBedAssignment.py” file, we build a constraint satisfaction/optimization model using Google OR-Tools' `cp_model` (Constraint Programming model) class. This library allows for the construction of constraint satisfaction models and provides a built-in solver that optimizes for a given objective function.

Our model includes variables for each bed at each minute of the day, each of which can be set to any one of the patient identifications from the list of patients. The model builds several constraints concerning each patient: the arrival time constraint ensures that each patient cannot be assigned a bed before they arrive at the hospital; the contiguity constraint ensures that a patient does not shift between beds, and implicitly ensures that a patient can only be assigned to one bed at a time; and the tenure constraints ensure that a patient is not assigned to a bed for any shorter or longer than their service time, unless their service time extends past midnight (in which case they are assigned until midnight). It is noteworthy that the greedy assignment solution also satisfies all the same constraints. Finally, the MIP solution forms an objective function that minimizes the sum of the wait time of each patient multiplied by inverse of CTAS priority level (which essentially translates to “patient cost”), applying a penalty of patient cost \* total minutes per day (1440 min) to patients that are not assigned.

When executed, the solver searches for solutions that satisfy all constraints, minimizing the objective function, and thus optimizing the bed assignment. We have also included user-entered parameters to set the number of beds, patients, and time limit of the MIP solver's execution. Note that if an optimal solution is not found before the time limit runs out, the solver displays the most optimal solution found so far.

To test the value and validity of the MIP and Greedy solutions to bed assignment, we repeatedly compared the results of the same set of patients and beds. Across various problem sizes, we performed this comparison with a new, random set of patients and beds at each denomination. The results are displayed in the next section of this report.

## B. Deep Learning

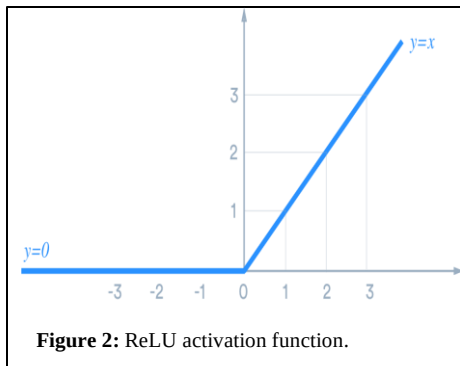


To automate the assignment of patient priority using triage symptom information to relieve the pressure for nurses making wrong decisions, a Deep Learning model assigns patient priority through this automated triage system.

Supervised machine learning has been chosen as the machine learning technique for this task because it uses labeled data to train algorithms to predict outcomes. The model is given labeled

training to learn the relationship between the input and the outputs. The two main types of supervised machine learning are classification and regression [7]. Classification was chosen for this task to perform multi-class classification task, where the model predicts which of 5 possible CTAS priority levels (1-5 seen in Figure 1) a patient is assigned [8]. The output is the priority levels (1-5), which is discrete. Therefore, the final layer of the algorithm will have 5 neurons. Common classification algorithms are linear classifiers, support vector machines (SVM), decision trees, k-nearest neighbor and random forest [7]. A Multi-Layer Perceptron (MLP) classifier was chosen as a model for this dataset because it can handle mixed data types and is scalable for large datasets.

*Hospital Triage and Patient History Data* is used for the deep learning aspect of *TreatmentFlow*, with this data loaded into the file and preprocessed. The data is first observed using the `load_data` function, and Pandas [9] is used to observe the file format. A graph is made to observe the given CTAS scores, and a bell curve with level 3 being the most common priority level is shown. The data is preprocessed by dropping columns from the data set that are non-medical demographic data, as to not contribute to any data leakage. The data in the *Hospital Triage and Patient History Data* csv file presents itself with numerical and categorical features which is split using Numpy [10] to ensure that the model is trained on high-quality data. Data is split into 80% training data and 20% test data sets with the input represented by 'x' (the columns of the file), and the target output represented by 'y' which are the CTAS values. The MLP classification algorithm that is being used is a Deep Neural Network (DNN) and is implemented using Scikit



Learn [11] with 3 hidden layers. To determine whether a neuron is activated or not the ReLU activation function (Figure 2) is used for the hidden layers. The ReLU activation function is the most common in deep learning and is a mathematical function to introduce non-linearity. If the input is positive the output is positive and if the input is negative the output is zero.

The model is evaluated using the testing data set (20% of the whole data set) which is saved in a confusion matrix to visualize the results [12].

### C. Bayesian Networks

*SymbiPredict* encodes symptoms as binary values (1 for presence, 0 for absence) and a final column indicating the confirmed diagnosis. We implemented a supervised machine learning approach to classify diseases based on patient-reported binary symptom data. A Bernoulli Naive Bayes classifier was chosen due to its suitability for binary feature data [11]. The model was trained using the training set ( $X_{train}$ ,  $y_{train}$ ) and a default smoothing parameter  $\alpha=1.0$ , leveraging the BernoulliNB implementation from scikit-learn. Model performance was assessed on the test set using classification accuracy and detailed metrics (precision, recall, and F1-score) derived from the `classification_report` and `confusion_matrix`. In addition, 5-fold cross-validation [11] was conducted on the full dataset to evaluate the model's generalizability. The mean and standard deviation of cross-validation scores were reported. To simulate clinical application, a prediction function was implemented that allows input of a dictionary of symptoms (present = 1, absent = 0). This function constructs a binary vector aligned with the training features and uses the trained model to predict the probability distribution across all known diseases. The top five most probable diseases are returned as a ranked list for the decision support tool. To identify the most influential symptoms for a given disease, the log-probability values learned by the Bernoulli Naive Bayes model were exponentiated and sorted in descending order. This provided insight into which symptoms were most indicative of each condition, based on the conditional probability:

$$P(\text{Diagnosis} \mid \text{Symptoms}) \propto P(\text{Diagnosis}) \times \prod P(\text{Symptom}_i \mid \text{Diagnosis})$$

Where  $P(\text{Diagnosis} \mid \text{Symptoms})$  represents the probability of a diagnosis given the observed symptoms,  $P(\text{Diagnosis})$  represents the prior probability of the diagnosis, and  $\prod P(\text{Symptom}_i \mid \text{Diagnosis})$  represents the conditional probability of observing each symptom given the diagnosis.

The diagnostic decision support tool generates a bedside document report containing a list of the top five most likely diagnoses, along with the observed symptoms that contributed to each diagnosis.

### III. Results

#### A. Constraint Satisfaction

**Table 1: Ratio of objective values** produced by the MIP approach and Greedy approach. For each cell, a ratio lower than one means the MIP approach produced a better solution than the Greedy Approach within the 120 second time limit on the solver. A cell value of “None” means the MIP solver produced no solution. (x axis: Patients, y axis: Beds)

|    | 5 | 10 | 20 | 50     | 100   | 250  |
|----|---|----|----|--------|-------|------|
| 5  | 1 | 1  | 1  | 2.6045 | 0.729 | None |
| 10 | 1 | 1  | 1  | 24344  | None  | None |
| 15 | 1 | 1  | 1  | inf    | None  | None |
| 20 | 1 | 1  | 0  | 84078  | None  | None |

**Table 2: Ratio of execution times** of MIP approach (with 120 second time limit on solver) and Greedy approach. (x axis: Patients, y axis: Beds)

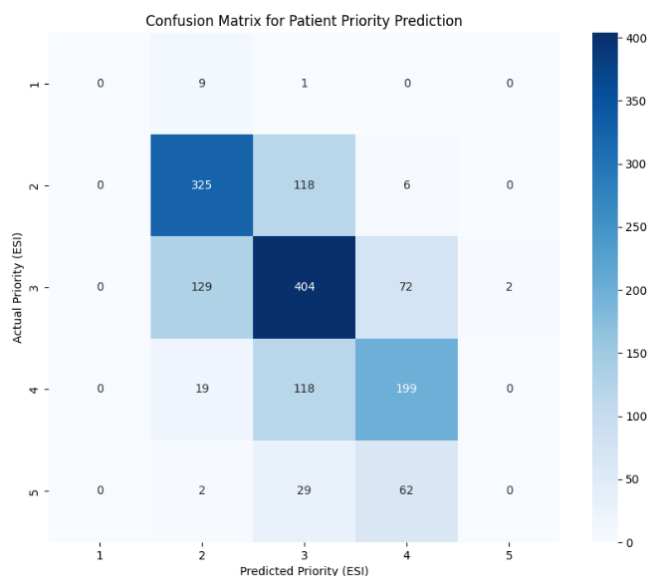
|    | 5     | 10    | 20     | 50     | 100   | 250   |
|----|-------|-------|--------|--------|-------|-------|
| 5  | 29015 | 33523 | 17991  | 58924  | 26433 | 18235 |
| 10 | 21085 | 59913 | 57062  | 100775 | 62713 | 24673 |
| 15 | 52333 | 67038 | 80641  | 111624 | 81349 | 34696 |
| 20 | 68041 | 88934 | 115251 | 119926 | 61192 | 45063 |

**Table 3: Greedy approach’s patients’ average wait time in minutes, and average number of unserved patients over 100 attempts.** (x axis: Patients, y axis: Beds)

|     | 5    | 10   | 20   | 50    | 100    | 250     | 500     |
|-----|------|------|------|-------|--------|---------|---------|
| 5   | 0, 0 | 0, 0 | 4, 0 | 51, 6 | 39, 66 | 33, 221 | 24, 471 |
| 10  | 0, 0 | 0, 0 | 0, 0 | 0, 0  | 40, 11 | 24, 188 | 33, 441 |
| 25  | 0, 0 | 0, 0 | 0, 0 | 0, 0  | 0, 0   | 30, 21  | 24, 345 |
| 50  | 0, 0 | 0, 0 | 0, 0 | 0, 0  | 0, 0   | 0, 0    | 24, 34  |
| 100 | 0, 0 | 0, 0 | 0, 0 | 0, 0  | 0, 0   | 0, 0    | 0, 0    |

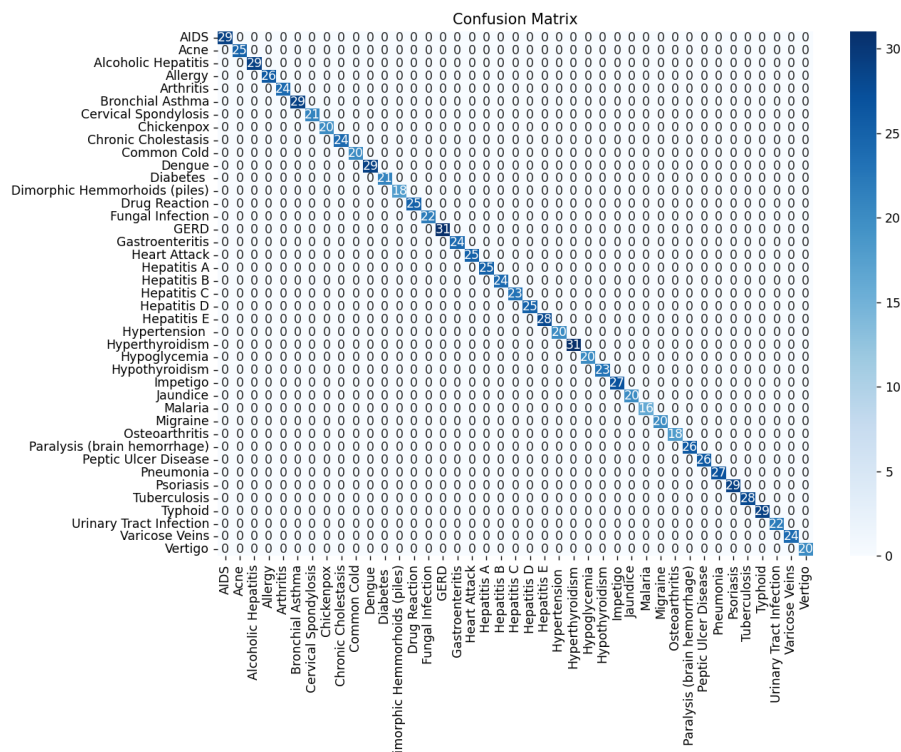


### B. Deep Learning



**Figure 3:** Confusion Matrix for analysis of DNN model predicting the CTAS values.

### C. Bayesian Networks



**Figure 4:** Confusion Matrix for Bayesian Network model for diagnosis prediction.

**Table 4:** Comparison of Test Diagnoses vs. Bayesian Network Predictions.

| Test # | Test Diagnosis (Google) | Bayesian Prediction  | Bayesian Probability | Correct Prediction |
|--------|-------------------------|----------------------|----------------------|--------------------|
| 1      | GERD                    | GERD                 | 0.7818               | Yes                |
| 2      | Peptic Ulcer Disease    | Peptic Ulcer Disease | 1.0000               | Yes                |
| 3      | AIDS                    | Malaria              | 0.9998               | No                 |
| 4      | Hypertension            | Hypertension         | 0.9542               | Yes                |
| 5      | Migraine                | Vertigo              | 0.8112               | No                 |
| 6      | Jaundice                | Jaundice             | 0.7368               | Yes                |
| 7      | Osteoarthritis          | Osteoarthritis       | 1.0000               | Yes                |
| 8      | Tuberculosis            | Bronchial Asthma     | 0.9988               | No                 |
| 9      | Cervical Spondylosis    | Cervical Spondylosis | 0.9986               | Yes                |
| 10     | Tuberculosis            | Heart Attack         | 0.8901               | No                 |
| 11     | Malaria                 | Malaria              | 0.5234               | Yes                |
| 12     | Heart Attack            | Heart Attack         | 1.0000               | Yes                |

## IV. Discussion

### A. Constraint Satisfaction

As seen in the comparison, tables 1 and 3 in the results section above, both the MIP and Greedy approaches can (usually) successfully reduce patient wait times. At a small scale (of patients and beds), the two methods return very similar results in terms of their objective values (Table 1) but differ greatly in their execution time (Table 3). The execution times of both methods increases with the problem size, but the execution time of the greedy solution remains under a single second, while the execution time of the MIP solution often takes multiple minutes. This is because building an MIP model with more variables leads to the creation of significantly more constraints, since they are dependent on the relationships between variables. For the results above, it is important to note that the solving time of each MIP model was limited to a maximum of 120 seconds. Without this limitation, testing each scale until proving optimality would become infeasible, as the default (unlimited) solving time increases exponentially, and thus the execution time ratios would be far greater. This increase of both approaches' execution time makes the ratio increase with the problem size, but it is limited by the 120 second time limit on the largest problems, which explains the reduction on the later columns of table 2.

MIP solving is notably faster when the number of patients is close to or lower than the number of beds, as the increased solution space allows for more instances of both feasible and optimal assignments. This is because there are more possible solutions that allow an optimal assignment of patients to beds. This increased chance for optimality comes from the fact that there are far more available resources at each individual minute, meaning that each patient is more likely to find an empty bed at their arrival time, and thus the MIP solver finds an optimal solution much faster. However, as the MIP solver more easily finds an optimal solution, the Greedy approach

also has fewer calculations to perform, which stabilizes the ratio of either approach's execution time in table 3.

When comparing each solution's performance in terms of their objective values, the two solutions often perform similarly, with some randomization involved. This randomization of relative performance depends on the layout of patient arrival times in each randomized list of patients. As discussed above, the MIP solver finds optimal solutions much more often when the number of patients is near the number of beds, or equal, or lower. However, the greedy solution provides near optimal solutions in these scenarios as well in a fraction of the time. As shown in table 1, the Greedy approach produced an equal or superior objective value in all but two observations. This shows that as problem size increases, the computational inefficiency of the MIP solution begins to outweigh the potential improvements it has in bed assignment optimization. This makes it impractical for large-scale settings, and thus impractical for most hospitals.

To conclude the comparison, the MIP model can sometimes find better solutions than the Greedy method, but the drastically larger computational complexity of the model makes it infeasible at scale. Since the differences between solutions found are marginal when limiting the solver's execution time, the Greedy bed assignment method, despite being a naïve solution, remains the superior bed assignment method. Using the Greedy bed assignment tool, we can reduce the logistical delays of patient wait times to negligible values, assuming a realistic number of patients compared to capacity (Table 3). The functional limit for the ratio of patients to beds that allows this tool to minimize wait times is when the ratio is less than ten. Since this is a very extreme potential ratio of patients to hospital resources, this tool remains very useful, even at scale. Thus, this application of constraint satisfaction creates a significant benefit for the operation of emergency departments.

### *B. Deep Learning*

The confusion matrix (Figure 3) is a table that is used to define the performance of a classification algorithm by comparing its predicted labels to the actual labels [12]. The y-axis of the matrix represents the true class, which are the actual labels, and the x-axis represents the predicted class, which are the model's output. The diagonal values from (1,1) to (5,5) represent the correct predictions, where the model's predicted CTAS priority matches the actual CTAS priority. The higher these values and the darker the coloring of the box, the better the model's accuracy. The values off the diagonal represent misclassifications, where the model predicted the wrong priority level. If a value is above the diagonal, it means the model underestimated priority. If a value is below the diagonal, it means the model overestimated priority. The model accuracy is 66% when trained on hospital data from our data set. The model seems to have the most difficulty distinguishing between priority 1 and 2, as well as 2 and 3, based on the number of misclassifications between them. There are very few cases in priority 1 and 5 which might indicate class imbalance.

Nurses' triage accuracy in assigning patient priority levels in a hospital setting typically ranges from 59% to 84.9%, with factors like experience, shift, and workload influencing the results [13]. Under-triage, where a patient's condition is underestimated, and over-triage, where it's overestimated, are common errors. The *TreatmentFlow* model is within these standards, however, future work includes developing a model with accuracy higher than 84.9% by finding the optimal hidden layer nodes to maximize patient priority accuracy.

### C. Bayesian Networks

The model achieved 100% accuracy on the test set when trained using data from our BN-specific dataset. As shown in the confusion matrix (Figure 4), all predictions were correct, with matches only along the diagonal—indicating perfect alignment between the model's predicted diagnoses and the actual diagnoses provided by the dataset. However, despite the perfect accuracy of the Naive Bayes classifier in this controlled evaluation, limitations persist in real-time diagnostic applications based on user-input symptoms.

The model's output includes a ranked list of differential diagnoses with associated probabilities, a table outlining symptom presence and importance, and a rationale for excluding less likely diagnoses. This structured, interpretable output supports clinical decision-making and demonstrates the transparency of our model. Such capabilities are critical for building trust among clinicians in our AI-assisted diagnosis tools.

To evaluate real-world applicability, we simulated a clinical diagnostic assessment on 12 fictitious patients, each with a unique disease profile. Symptom input was sourced from [14], and diagnosis is summarized in Table 4. Our diagnostic tool correctly identified the disease in 8 out of the 12 cases, with predicted probabilities ranging from 0.5234 to 1.0000.

Several factors may explain the misdiagnoses and the probabilities. Firstly, the Bayesian Network does not incorporate extrinsic patient information which is contextual data on external influences, such as geographic location or lifestyle factors. For instance, test number 3 (Table 4), the model predicted Malaria with 99% probability when symptoms for AIDS were input. This misclassification likely stems from overlapping symptoms; however, if additional patient information had been available, the Bayesian model would adjust its prediction based on its different weightings. The model could better distinguish between the two diagnoses if, for this example, recent travel to malaria-endemic areas or high-risk sexual behavior was available as a feature.

Secondly, as this tool is intended to support, not replace, clinical judgment, further diagnostic tests are necessary to confirm or rule out conditions. For example, for patient number 11 (Table 4), the Bayesian probability of the Malaria diagnosis is only 52%. The second-ranked diagnosis, Hypertension, had a probability of 45%, as seen in the bedside document. A clinician could use

laboratory tests such as blood films (the gold standard for Malaria) [15], blood pressure readings or ECGs (for Hypertension) to confirm or exclude each diagnosis.

Future work includes incorporating additional extrinsic patient information such as found in *Hospital Triage and Patient History Data*, into the Bayesian Network model. Incorporating *all* the available triage information and Boolean symptom information from our deep learning module would allow the Bayesian model to adjust symptom weights dynamically by adding new patient-specific nodes. This enhancement is expected to improve diagnostic accuracy further.

Notably, the model performs better when symptoms are more uniquely associated with a specific condition. Although some overlapping symptoms can be present across differential diagnoses, the algorithm accurately weighted their relative importance and cross-referenced them with the complete symptom profile. It is evident that the patient - doctor communication must be strong for the doctor to input the correct symptoms in order to have a strong diagnosis.

## V. Conclusion

AI-driven triage systems like *TreatmentFlow* have the potential to revolutionize emergency room efficiency by addressing key challenges in patient prioritization, bed assignment, and diagnostic support. By leveraging Deep Learning for automated triage, Constraint Optimization for bed allocation, and Bayesian Networks for diagnostic assistance, *TreatmentFlow* aims to reduce emergency wait times, alleviate stress on healthcare workers, and improve patient health outcomes. As hospitals continue to integrate AI technologies, solutions such as this have the potential to play a critical role in optimizing resource management and ensuring timely, effective care for all patients.

To facilitate the demonstration of *TreatmentFlow*'s capabilities, we developed a streamlined module, *TreatmentFlow\_Lite*, in which the three components of the AI system are tightly integrated, allowing the output of one module to serve as the input for the next. Specifically, from a randomly generated set of real patients from our dataset, the associated CTAS priority levels predicted in the Deep Learning section are passed into the Constraint Optimization section to optimally assign these patients to beds, and once the beds are assigned, the present patient data can be passed into the Bayesian Network section to predict diagnoses for a doctor-aimed bedside document associated with each patient. Aligning all three of these components allows for a seamless assistive tool for healthcare workers to address patients' movement and treatment throughout the hospital. *TreatmentFlow* demonstrates that a thoroughly developed system that aligns with these goals and methodologies, has the chance to make a legitimately strong impact in healthcare.

In future iterations, *TreatmentFlow* can be validated through simulation environments and prospective clinical trials. Success for this project is defined by its ability to outperform traditional triage in both speed and accuracy, reduce average emergency department wait times through

patient prioritization and routing, and improve diagnostic precision. The system aims to ease the workload of clinical staff by offering automated, interpretable recommendations that support rather than replace human judgment. Additionally, by testing its functionality in real-world medical settings, the impact of these technologies can be measured.

## References

- [1] System performance. Emergency Department Time Spent by Patients in Ontario – Health Quality Ontario (HQO). (n.d.). <https://www.hqontario.ca/system-performance/time-spent-in-emergency-departments>
- [2] Peta, D., Day, A., Lugari, W., Gorman, V., Ahayalimudin, N., Pajo, V. (2023, November 2). Triage: A global perspective. *Journal of Emergency Nursing*.  
<https://www.sciencedirect.com/science/article/abs/pii/S0099176723002143#:~:text=Triage%20is%20a%20process%20by,timely%20access%20to%20emergency%20care>.
- [3] Government of ontario | gouvernement de l'ontario. ontario.ca. (n.d.).  
[https://files.ontario.ca/moh\\_3/moh-manuals-prehospital-ctas-paramedic-guide-v2-0-en-2016-12-31.pdf](https://files.ontario.ca/moh_3/moh-manuals-prehospital-ctas-paramedic-guide-v2-0-en-2016-12-31.pdf)
- [4] Emergency room triage: How does it work? - the daily scan. (n.d.).  
<https://thedailyscan.providencehealthcare.org/2018/11/emergency-room-triage-how-does-it-work/>
- [5] Mafaufau, M. (2019, June 3). Hospital triage and patient history data. Kaggle.  
<https://www.kaggle.com/datasets/maalona/hospital-triage-and-patient-history-data>
- [6] Tucker, J. (2024, April 2). Symbipredict. Mendeley Data.  
<https://data.mendeley.com/datasets/dv5z3v2xyd/1>
- [7] Belcic, I., & Stryker, C. (2024, December 26). What is supervised learning?. IBM.  
<https://www.ibm.com/think/topics/supervised-learning#:~:text=its%20training%20data-,Types%20of%20supervised%20learning,next%20layer%20in%20the%20network>.
- [8] Emscimprovement. Emergency Nurses Association. (2002).  
[https://media.emscimprovement.center/documents/Emergency\\_Severity\\_Index\\_Handbook.pdf](https://media.emscimprovement.center/documents/Emergency_Severity_Index_Handbook.pdf)
- [9] Pandas. pandas. (n.d.). <https://pandas.pydata.org/>
- [10] NumPy. (n.d.). <https://numpy.org/>
- [11] Learn. scikit. (n.d.). <https://scikit-learn.org/stable/>
- [12] Confusion matrix. Confusion Matrix - an overview | ScienceDirect Topics. (n.d.).  
<https://www.sciencedirect.com/topics/engineering/confusion-matrix#:~:text=A%20confusion%20matrix%20is%20a,malignant%20tissue%20is%20considered%20cancerous>.
- [13] Suamchaiyaphum, K., Jones, A. R., & Markaki, A. (2024). Triage Accuracy of Emergency Nurses: An Evidence-Based Review. *Journal of emergency nursing*, 50(1), 44–54.  
<https://doi.org/10.1016/j.jen.2023.10.001>

[14] Mayo Foundation for Medical Education and Research. (n.d.). Top-ranked hospital in the nation. Mayo Clinic. <https://www.mayoclinic.org/>

[15] Centers for Disease Control and Prevention. (n.d.). Malaria diagnostic tests. Centers for Disease Control and Prevention. <https://www.cdc.gov/malaria/hcp/diagnosis-testing/malaria-diagnostic-tests.html>